Lab 1 - Credit Card Fraud Detection

Chan Lee

April 17, 2025

1 Introduction

In this report, we seek to understand what factors best predict the occurrence of credit card fraud. This is especially important to credit lenders within the financial industry, as they are generally the ones who incur the cost of credit card fraud. We purport that this anaylsis will help these lenders take the crucial steps to mitigate their losses. Of course, this issue also inconveniences the credit borrowers. Hence, we hope this analysis can help them to better understand how to avert this crisis in the future as well.

2 The Data

This data was presumably synthesized by Dr. Bradly Stadie. Thus, they do not represent the information of any real person, place, or thing.

2.1 Data Collection

The following variables were used in our analysis. We excluded variables with no data values. We also excluded variables that are mere copies of other variables by eliminating columns with the same column names and checking among columns with different names those with duplicate values in all rows. After doing this, we are left with 22 variables (including the predictor variable *isFraud*).

1. accountNumber: a unique identifier for the customer account associated with the transaction
2. creditLimit: the maximum amount of credit available to the customer on their account
3. availableMoney: the amount of credit available to the customer at the time of the transaction
4. transactionDateTime: the date and time of the transaction
5. transactionAmount: the amount of the transaction
6. merchantName: the name of the merchant where the transaction took place
7. acqCountry: the country where the acquiring bank is located
8. merchantCountryCode: the country where the merchant is located
9. posEntryMode: the method used by the customer to enter their payment card information during the transaction
10. posConditionCode: the condition of the point-of-sale terminal at the time of the transaction
11. merchantCategoryCode: the category of the merchant where the transaction took place
12. currentExpDate: the expiration date of the customer's payment card
13. accountOpenDate: the date the customer's account was opened
14. dateOfLastAddressChange: the date the customer's address was last updated

15. cardCVV: the three-digit CVV code on the back of the customer's payment card
16. enteredCVV: the CVV code entered by the customer during the transaction
17. cardLast4Digits: the last four digits of the customer's payment card
18. transactionType: the type of transaction
19. currentBalance: the current balance on the customer's account
20. cardPresent: whether or not the customer's payment card was present at the time of the transaction
21. expirationDateKeyInMatch: whether or not the expiration date of the payment card was entered correctly during the transaction
22. isFraud: whether or not the transaction was fraudulent

2.2 Data Cleaning

2.2.1 Outliers

We first began removing the outliers, which was a nontrivial process. Here's an assessment of the outliers for each of the numerical variables:

- *accountNumber, cardCVV, enteredCVV,cardLast4Digits* have no outliers, and even if there were we wouldn't remove any, since these numbers are arbitrarily chosen for the credit card owner and have no meaning.
- The variable creditLimit has 48,781 values equal to 50,000, which is significantly higher than the rest of the data, which ranges from 0 to 20,000. Although these values are statistical outliers, they appear to represent a meaningful and frequent upper limit in the dataset rather than errors or anomalies. Given both the large number of occurrences and the potential domain significance of the 50,000 value, we choose not to cap or remove these outliers, as doing so could distort the underlying distribution and business interpretation.
- *availableMoney* is heavy right skewed with 30,092 outliers. We will use the Tukey Method to remove these.
- *transactionAmount* has only 10,000 outliers. We will remove these, and focus our analysis only on the overwhelmingly large number of individuals smaller transactionAmount values. We will use the Tukey Method to remove these.
- *posEntryMode* mode has around 35,000 outliers all the way in the right. We will use the Tukey Method to remove these.
- *posConditionCode* has a whopping 160,000 outliers according to the Tukey Method, but this number is deceivingly large because most of the data points have values 1 and 8, and so the Tukey Method considers those with value 8 as outliers (see histogram). However, since an overwhelming majority of data points take on either 1 or 8 as its *posConditionCode* value, we will not exclude any of these points. Instead, we will only exclude the 10,000 outliers that take on a value of 99.
- currentBalance - heavily right skewed, with around 21,808 outliers. We will remove these via the Tukey Method.

Please refer to the histograms to see the distribution of the aforementioned variables. Note that our removal of outliers resulted in the number of data points to go from 786,363 to 579,743.

2.2.2 Imputing Missing Values

For the variables *merchantCountryCode* and *acqCountry*, *posEntryMode*, *transactionType*, and *posConditionCode*, we impute missing dates with a "missing" category, because the missingness gives information.

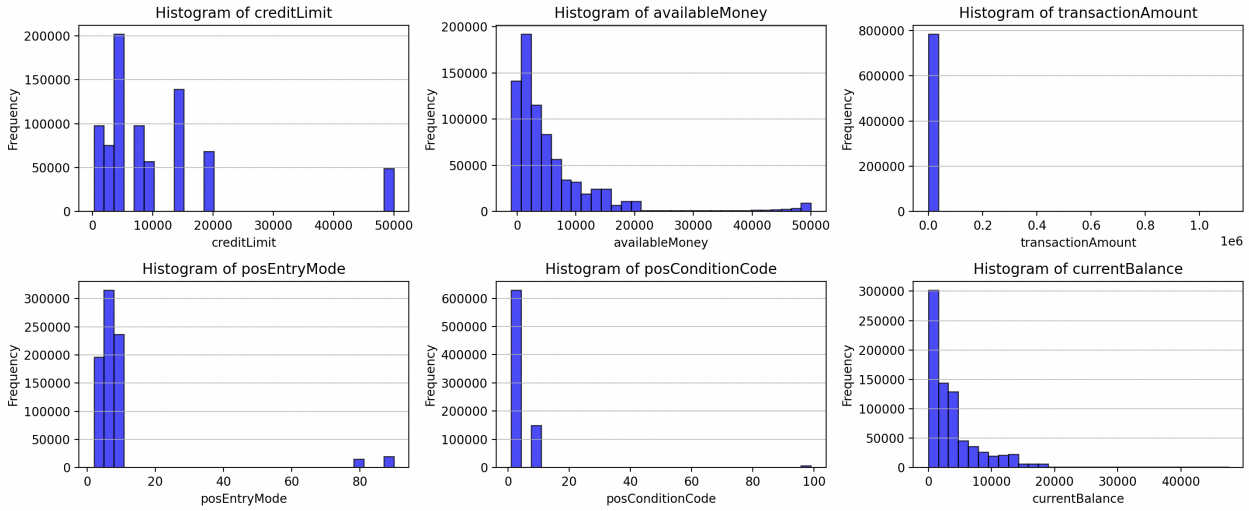


Figure 1: Histograms

2.2.3 Cleaning time variables

We find the time variables *'transactionDateTime', 'currentExpDate', 'accountOpenDate', 'dateOfLastAddressChange',* and *'expirationDateKeyInMatch'*, and converted them to datetime format using the to_datetime() function in the pandas package. We then extracted the meaningful features for each of the time variables (the year, month, day, hour, or weekday) if such information was available, and made them into new columns. For example, the variable *transactionDateTime_year* extracts the year that the transactions took place from *transactionDateTime*, which tells us the year, month, day, and time when that transaction took place.

2.2.4 Further data wrangling for ID variables

We found that the *cardCVV, enteredCVV, cardLast4Digits* variables sometimes didn't store leading zeroes, so they were shorter in length than they were supposed to be. Hence, we created separate variables *cardCVV_str, enteredCVV_str, cardLast4Digits_str* corresponding to the original three variables respectively, which represents the numbers in string format and contains the leading zeroes.

3 Regression Analysis

3.1 The Relationship between isFraud and cardCVV, enteredCVV, and cardLast4Digits

As seen in 2, credit cards with cardCVV and enteredCVV number 869 have significantly higher fraud counts than those of different values. This is a very interesting result, since CVV's are chosen almost randomly, making it very hard for a someone to guess someone else's CVV number based on other pieces of information. In other words, because the algorithm that determines the CVV is very complex and relatively unknown, we can essentially say that CVV's are chosen at random. Hence, we would expect fraud counts among CVV numbers to be pretty uniform, which is not the case here. The same thing is true for a card's last 4 digits. From 2, we see that cards with 0593 as their last four digits have significantly more fraud counts, while the rest are relatively uniform. This is also very unexpected, since we expect fraud counts to be relatively uniform among all last four digit combinations.

3.2 Distribution of *transactionAmount*

We see the *transactionAmount* is extremely right-skewed, with most data points having a transaction amount equal to zero, with most having less than 400, as seen in 3 and 4. It is interesting to see that there is a wide range of unique transaction amount values, ranging from zero to $1.0e6$.

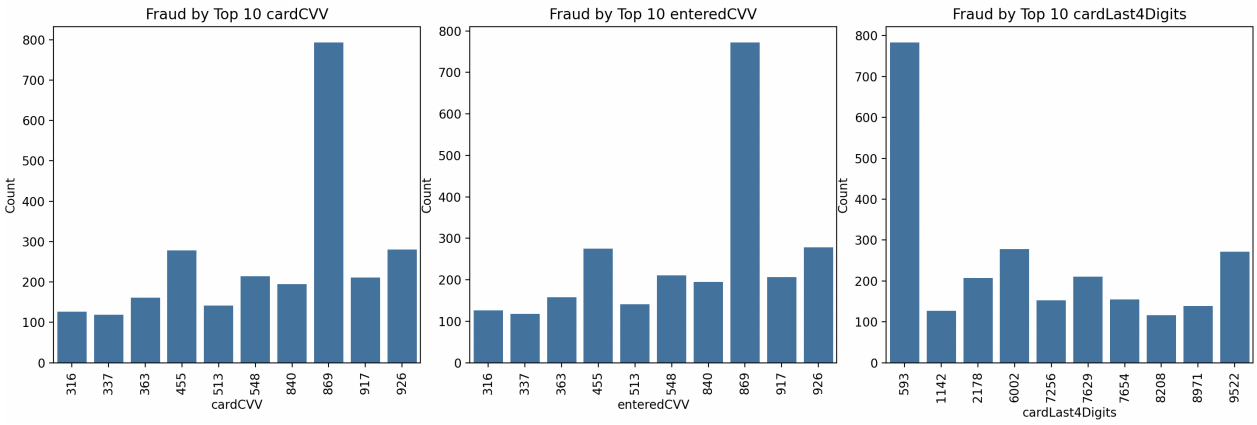


Figure 2: Relationship between isFraud and cardCVV, enteredCVV, and cardLast4Digits

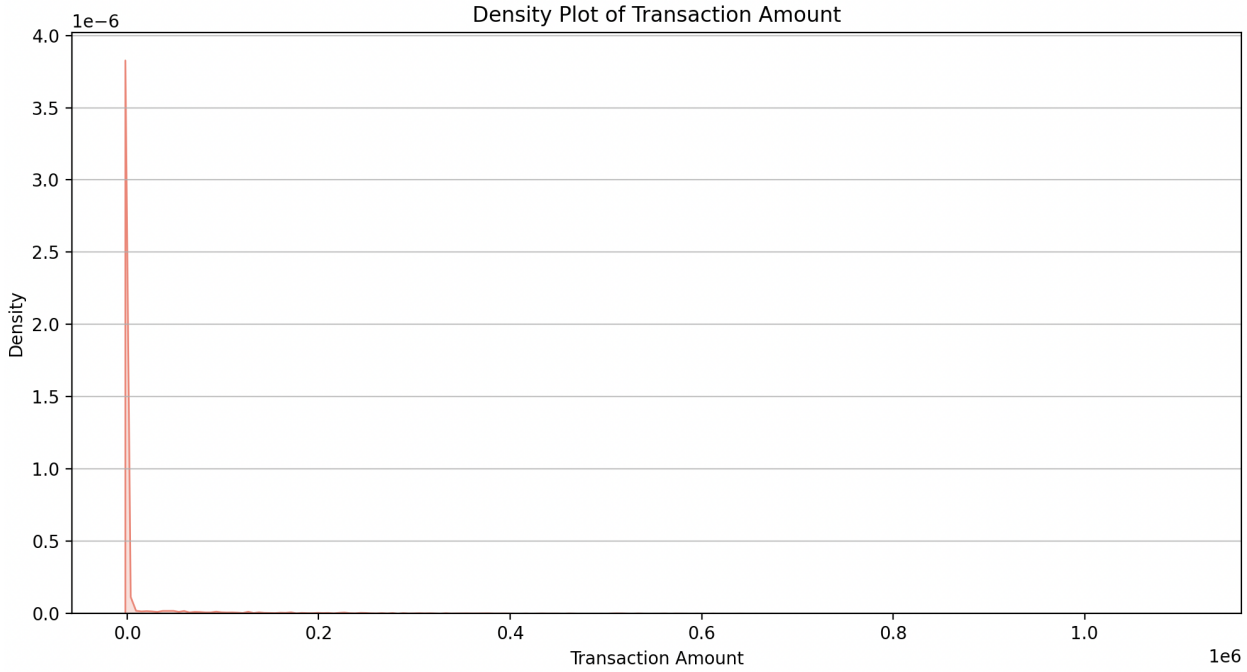


Figure 3: Density Plot of *transactionAmount*

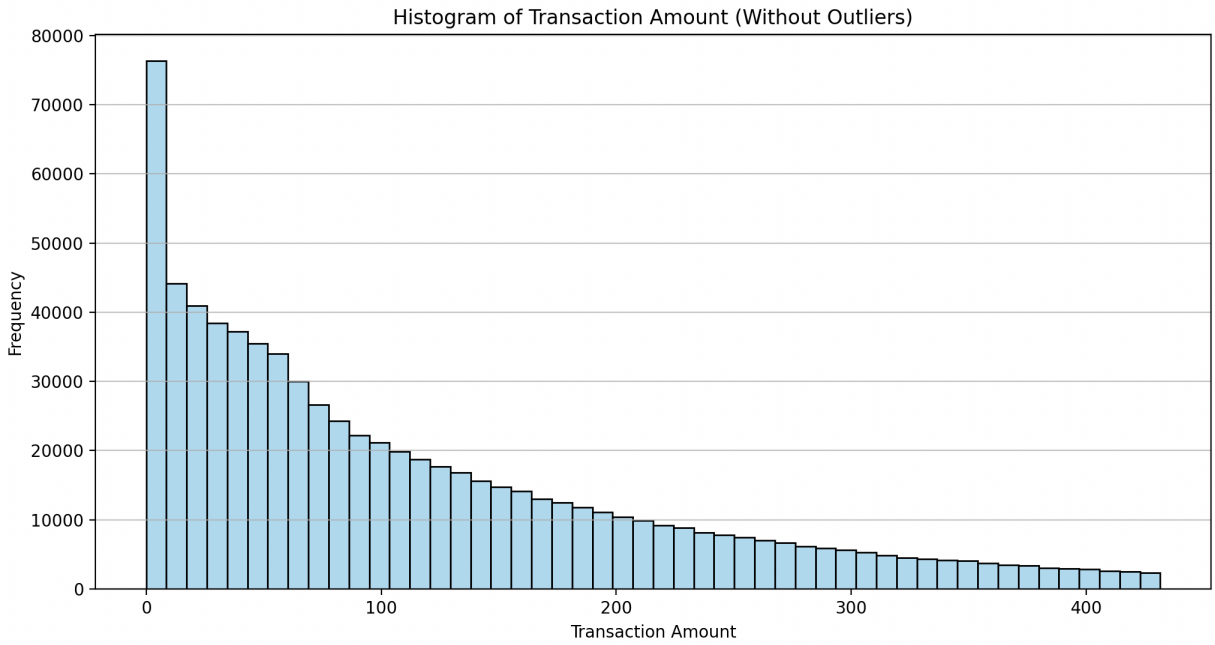


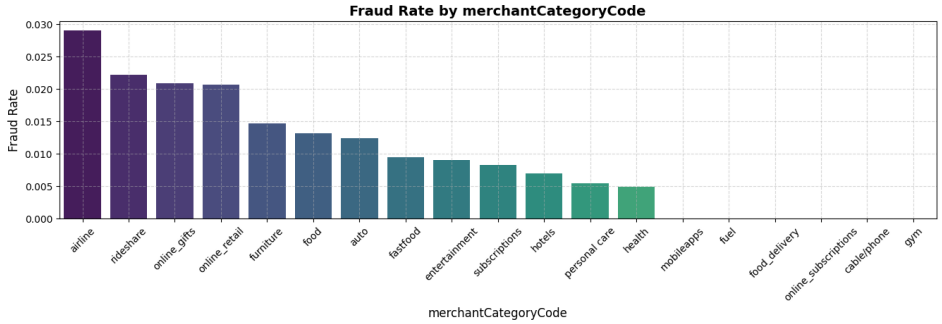
Figure 4: Density Plot of *transactionAmount* w/o Outliers

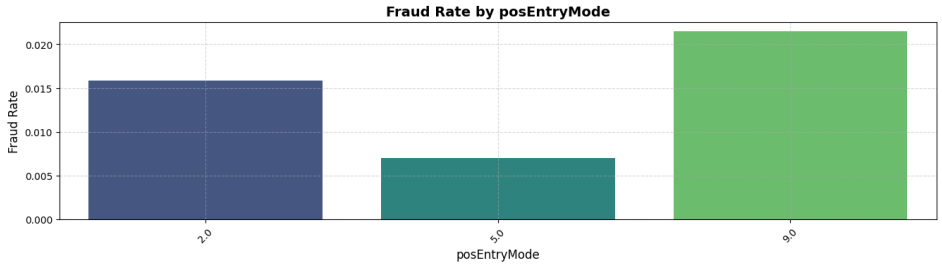
3.3 Relationship between *isFraud* and categorical variables

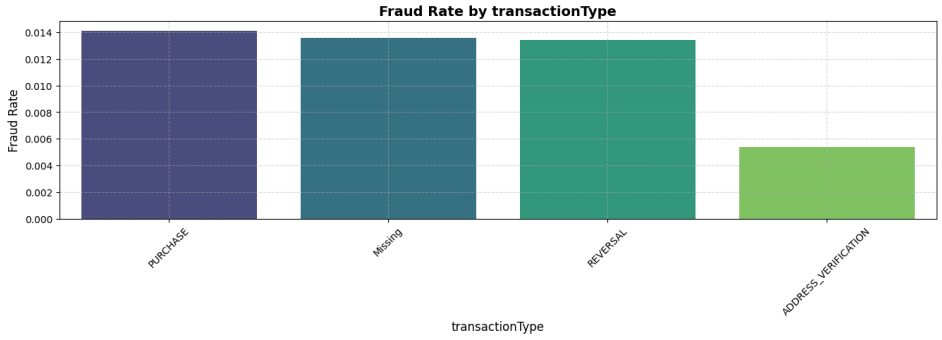
- We can see in 5 that fraud rate is most frequent in airlines and during rideshares, which makes sense given how busy these places can get. Online retail and online gifts follows, which also makes sense since people can steal others' credit card information via hacking.
- We can see in 5 that method 9.0 for entering in the credit card information led to the most fraud, followed by 2.0, and 5.0
- Next, we see in 5 that the transaction type categories Purchase, Reverse, and Missing had the most fraud, while address verification had less than half of each of the former two. This is probably because those who commit fraud often do not know the address of those who they are defrauding, so this is a relatively safe transaction type. It is interesting to see that there is a significant number of missing data points: this probably points to the possible correlation between missingness and credit card fraud.
- We can see that fraud rates for the 1.0 posConditionCode category were the highest, followed by 8.0.
- Lastly, the missing data points had the highest fraud rates compared to the other categories in merchantCountryCode.

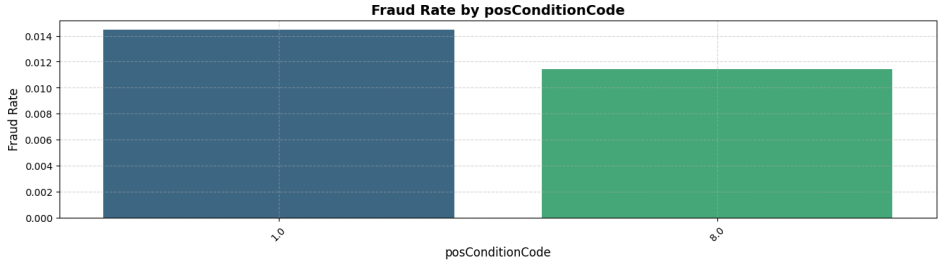
3.4 Relationship between *isFraud* and *transactionType* conditioned on *merchantCategoryCode*

As we can see in 6, when we condition on *merchantCategoryCode*, and explore the relationship between *isFraud* and *transactionType*, typically the PURCHASE category and MISSING category have the highest fraud rates. In addition, in fastfood restaurants and when buying online gifts, we see that there are a lot of missing variables tha that have high fraud rates, suggesting that these are settings under which cards can go missing and eventually the card ownder defrauded.









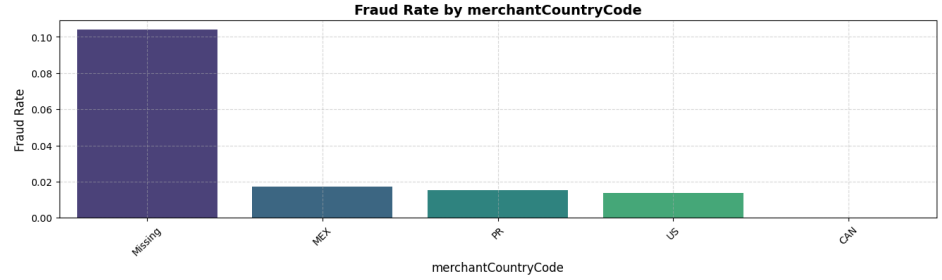


Figure 5: Relationship between *isFraud* and categorical variables

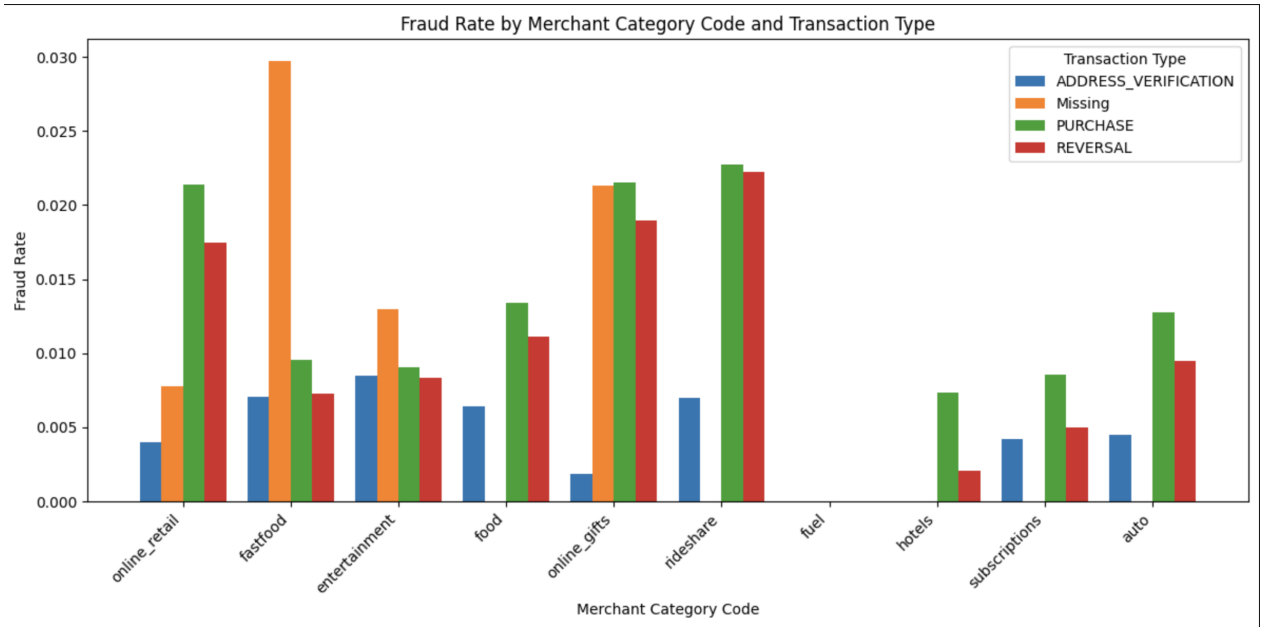


Figure 6: Relationship between *isFraud* and *transactionType* conditioned on *merchantCategoryCode*

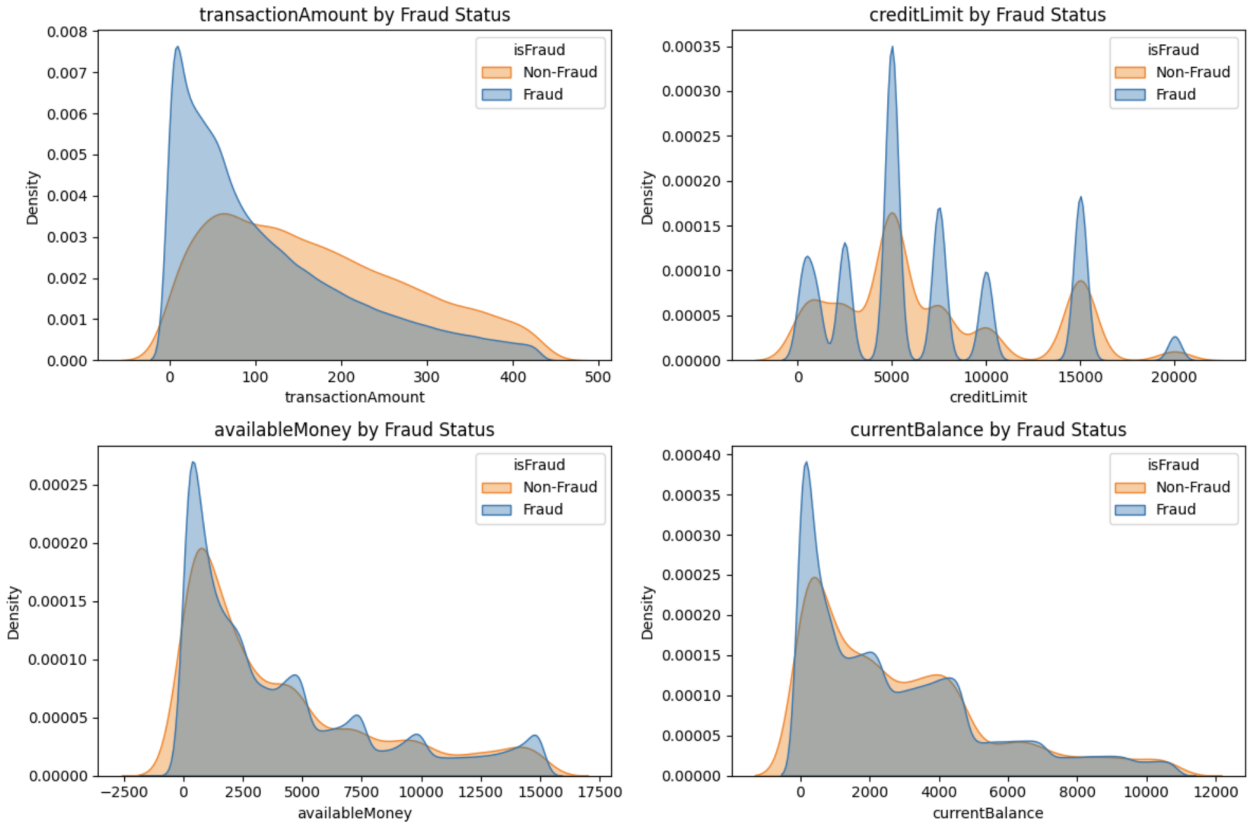


Figure 7: Conditional Probability Density Plots

3.5 Conditional probability density plots (or other suitable visualizations) for the numerical variables

As we can see in 7, lower transaction amounts tend to have much greater fraud rates compared to higher transaction amounts, which might be because those who tend to use their cards more tend to be more aware of where their card is and what they use it for. We can also see that the fraud rates for credit limit are multimodal, because there are different families of credit cards, each with different credit limits (i.e. credit limits aren't uniformly distributed, but are more clustered). We also see that those with lower available money tend to fall victim to credit card fraud way more often than those with more available money. This may also be because those who use their credit card tend to care less about it, while those who use it more tend to know where their card is at all times and have their personal finances in order. We also see that the same is true for currentBalance, where those with smaller currentBalances tend to have higher rates of fraud.

4 Discussion (Q11-13)

4.1 Question 11

For this analysis, we defined a "multi-swipe" event as any transaction sharing the same cardLast4Digits and identical transactionAmount that occurs within 60 seconds of a prior swipe, and we excluded the very first (normal) swipe in each such sequence. Applying these criteria to 579,743 transactions, we identified 3,277 extra swipes—just 0.57% of all transactions—that meet our multi-swipe definition. In dollar terms, these extra swipes account for $394,114.62 out of a total $65,483,358.79 processed, or 0.60% of the overall transaction volume.

Although multi-swipe events represent well under 1% of all transactions (0.57%), they account for a slightly larger share of dollar volume (0.60%), indicating that repeat swipes tend to occur on transactions that are, on average, marginally higher in value than the typical sale. Because we required the same card and identical amount within just 60 seconds, these extra swipes almost certainly reflect either customer or terminal retries—perhaps chip-reads that didn't register cleanly or tap-and-go attempts at busy self-checkout kiosks. The fact that volume share slightly exceeds count share suggests these hiccups are more common on mid- to high-ticket items, where cardholders or terminals are less tolerant of initial read failures. Although rare, these "double-swipes" can still total nearly $400 K in processed volume over our sample, hinting at a measurable impact on both customer experience (e.g., potential duplicate authorizations) and back-office reconciliation. Targeting the merchant categories, entry modes, or terminal conditions most prone to rapid retries could therefore be a fruitful next step to streamline checkout, reduce chargeback risk, and improve overall throughput.

4.2 Question 12

The dataset exhibits a pronounced class imbalance, with 571,700 non-fraud transactions (98.61%) versus just 8,043 fraud cases (1.39%), or roughly one fraudulent event per seventy-one legitimate ones. Such skew means that a naïve classifier predicting "no fraud" would achieve 98.6% accuracy yet catch none of the actual frauds, so accuracy alone is misleading. Instead, modeling efforts must emphasize recall for the fraud class—so as to detect as many true frauds as possible—while also monitoring precision to avoid inundating analysts with false positives. To address the imbalance, one might upsample the minority class (e.g., via SMOTE), downsample the majority class, or leverage class-weighted algorithms that penalize mistakes on fraud more heavily. Stratified cross-validation will ensure representative sampling, and careful decision-threshold tuning will balance the cost trade-off between missing fraud and triggering false alarms. In settings where fraud is exceptionally rare, exploring anomaly-detection methods may further enhance the ability to flag anomalous transactions.

4.3 Question 13

One simple yet effective approach is to apply random oversampling of the minority (fraud) class so that it matches the majority (non-fraud) class. After oversampling, the fraud class has been boosted from 8,043 (1.39%) up to 571,700 (50%), yielding a perfectly balanced 1:1 ratio.

Balancing the classes in this way generally improves a model's ability to learn the minority patterns, leading to higher recall on fraud cases, but it can also introduce a risk of overfitting to duplicated examples and inflate false

positives. In practice, one often combines oversampling (or SMOTE) with robust cross-validation, threshold tuning, and possibly ensemble methods or cost-sensitive learning to maximize fraud detection while keeping the false-alarm rate at an acceptable level.

5 Conclusion

This lab offered us the opportunity to understand the different steps for cleaning the data and the different kinds of interesting analysis we can do to different combinations of variables. We also learned about ways to mitigate class imbalance in the categorical predictor response.

6 Acknowledgements

I worked mostly alone, but discussed with Qijie Zhu and Yu Xiao.